```
from openai import OpenAI
import numpy as np
import json

client = OpenAI(api_key = "sk-proj-hDJsiqMQMh93vHQe_YizsTuU15PNyToZ0GcY1fVRVfNij2f_g57Nh-Bv4b-GqHPAj-1V4u9dsqT3B1bkFJY9J8b3HzYbEfMyx
```

helper: format obervation history into a prompt

```
def build_observation_context(X_sample, Y_sample):
    """turn array into readable sentences"""
    lines = ["Past observations (x->y):"]
    for i in range(len(X_sample)):
        x_val = X_sample[i][0] if X_sample.ndim > 1 else X_sample[i]
        y_val = Y_sample[i][0] if Y_sample.ndim > 1 else Y_sample[i]
        lines.append(f"x={x_val:.4f}, y={y_val:.4f}")
    return "\n".join(lines)

def llm_propose_location(X_sample, Y_sample, bounds, n_candidates=5):
    """Ask the LLM to propose next x to evaluate"""
    context = build_observation_context(X_sample, Y_sample)
    x_min, x_max = bounds[0][0], bounds[0][1]

    prompt = f"""You are helping optimize a black-box function using Bayesian Optimization.
    {context}
    The search space is x in [{x_min}, {x_max}].
    Your goal: suggest the single x value most likely to give a HIGH y value.
    Think about:
    - Where have we not explored yet? (exploration)
    - Where did we see high y values? (exploitation)

    Respond ONLY with a JSON object, no explanation:
    {{
        "x": <float>, "reasoning": "<one line>"
    }}"""

    response = client.chat.completions.create(
            model = "gpt-4o",
            max_tokens = 200,
            messages = [{
                    "role": "user",
                    "content": prompt
            }],
            response_format = {"type": "json_object"}
    )

    result1 = json.loads(response.choices[0].message.content)
    x_next = float(result1["x"])
    x_next = np.clip(x_next, x_min, x_max)
    print(f"LLM proposes x={x_next:.4f} | reasoning: {result1['reasoning']}")
    return np.array([[x_next]])

def llm_score_candidates(X_candidates, X_sample, Y_sample):
    context = build_observation_context(X_sample, Y_sample)
    candidates_str = ",".join([f"{x[0]:.4f}" for x in X_candidates])
    prompt = f"""You are a surrogate for Bayesian Optimization.

    {context}

    Rate each of these candidate x values by how likely they are to have HIGH y.
    Candidates: [{candidates_str}]

    Respond ONLY with a JSON aray of scores (floats between 0 and 1),
    same order as candidates, no explanation:
        {{"scores": [0.2, 0.8, 0.5, 0.1, 0.9]}}"""

    response = client.chat.completions.create(
            model = "gpt-4o",
            max_tokens = 200,
            messages = [{
                    "role": "user",
                    "content": prompt
            }],
            response_format = {"type": "json_object"}
    )


    result2 = json.loads(response.choices[0].message.content)
    scores = result2["scores"]
```

```
    return np.array(scores)
```

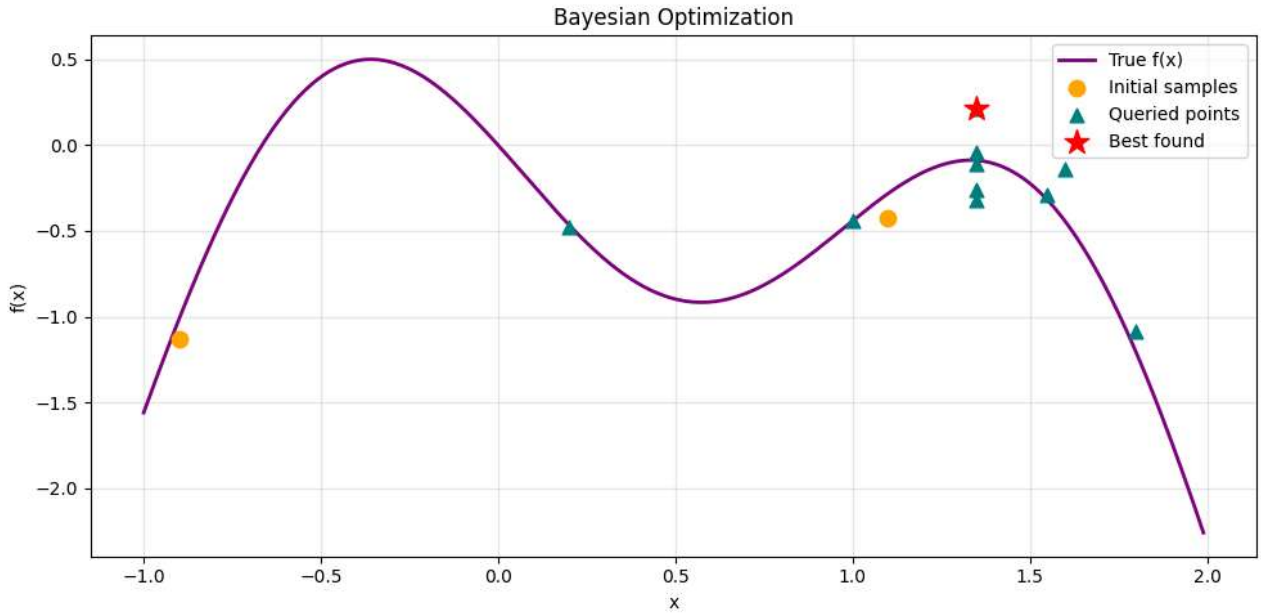
```
bounds = np.array([[-1.0, 2.0]]) # one parameter with a bound
noise = 0.2

def f(X, noise=noise):
    return -np.sin(3*X) - X**2 + 0.7*X + noise* np.random.randn(*X.shape)

X_init = np.array([[-0.9], [1.1]])
Y_init = f(X_init)

# real value and function
X = np.arange(bounds[:, 0], bounds[:, 1], 0.01).reshape(-1, 1)
Y = f(X, 0) # without noise
```

```
/tmp/ipykernel_26659/1372488964.py:11: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will erro
  X = np.arange(bounds[:, 0], bounds[:, 1], 0.01).reshape(-1, 1)
```

```
X_sample = X_init.copy()
Y_sample = Y_init.copy()
n_iter = 10

for i in range(n_iter):
    X_candidates = llm_propose_location(X_sample, Y_sample, bounds, n_candidates=5)
    scores = llm_score_candidates(X_candidates, X_sample, Y_sample)
    best_idx = np.argmax(scores)
    X_next = X_candidates[[best_idx]] # shape(1,1)
    Y_next = f(X_next, noise)
    print(
            f"Iter {i+1}: proposed {len(X_candidates)} candidates, "
            f"picked x={X_next[0][0]:.4f} (score={scores[best_idx]:.4f}), "
            f"y={Y_next[0][0]:.4f}"
    )
    X_sample = np.vstack((X_sample, X_next))
    Y_sample = np.vstack((Y_sample, Y_next))
```

```
LLM proposes x=1.5500 | reasoning: Balance exploration around the higher y value region.
Iter 1: proposed 1 candidates, picked x=1.5500 (score=0.3000), y=-0.2905
LLM proposes x=1.8000 | reasoning: Exploration in the untested higher region of the search space, away from observed lower y values.
Iter 2: proposed 1 candidates, picked x=1.8000 (score=0.7000), y=-1.0869
LLM proposes x=1.3500 | reasoning: Balance exploration between 1.1 and 1.55 where highest y values were observed.
Iter 3: proposed 1 candidates, picked x=1.3500 (score=0.7000), y=-0.2628
LLM proposes x=1.3500 | reasoning: This value previously yielded the highest y, suggesting further exploitation around it.
Iter 4: proposed 1 candidates, picked x=1.3500 (score=0.9000), y=-0.0413
LLM proposes x=0.2000 | reasoning: Combine exploration (unprobed region) and exploitation (near previous max at x=1.35).
Iter 5: proposed 1 candidates, picked x=0.2000 (score=0.5000), y=-0.4746
LLM proposes x=1.3500 | reasoning: Exploitation where the highest y was observed.
Iter 6: proposed 1 candidates, picked x=1.3500 (score=0.8000), y=-0.1114
LLM proposes x=1.0000 | reasoning: 1.0 is unexplored and near regions where higher y values were observed, balancing exploration and exp
Iter 7: proposed 1 candidates, picked x=1.0000 (score=0.7000), y=-0.4399
LLM proposes x=1.3500 | reasoning: Exploitation: multiple trials at x=1.35 had relatively high y values.
Iter 8: proposed 1 candidates, picked x=1.3500 (score=0.1000), y=0.2131
LLM proposes x=1.3500 | reasoning: Previous evaluations at x=1.3500 have yielded the highest y value, suggesting a promising area for ex
Iter 9: proposed 1 candidates, picked x=1.3500 (score=0.7000), y=-0.3220
LLM proposes x=1.6000 | reasoning: Exploration: unexplored area near 1.55, which showed relatively high y values.
Iter 10: proposed 1 candidates, picked x=1.6000 (score=0.3000), y=-0.1435
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.plot(X, Y, 'purple', lw=2, label='True f(x)')
plt.scatter(X_init, Y_init, c='orange', s=80, zorder=5, label='Initial samples')
plt.scatter(X_sample[2:], Y_sample[2:],
                        c='teal', s=60, marker='^', zorder=5, label='Queried points')
best_idx = np.argmax(Y_sample)
plt.scatter(X_sample[best_idx], Y_sample[best_idx],
                        c='red', s=200, marker='*', zorder=6, label='Best found')

plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Bayesian Optimization')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



开始借助 AI 编写或生成代码。